深度学习基础

第 9 讲 — 注意力机制与 Transformer 架构

孟翰

人工智能学院
中国石油大学（北京）

2026 年春季学期

本讲内容

1. 回顾与本讲目标
2. 从信息瓶颈到注意力
3. 缩放点积注意力
4. Self-Attention：让序列自己看自己
5. 多头注意力
6. Transformer Encoder 完整架构
7. Decoder 与完整 Transformer
8. 代码实战与现代联系
9. 小结与下讲预告

第 8 讲我们学到

RNN/LSTM/GRU 用**隐状态** $\boldsymbol{h}_t$ 建模序列，门控机制缓解了梯度问题。但任何基于“串行递推”的结构都有三个根本性局限：

三个痛点（记住它们，今天一个个解决）

1. **无法并行**：必须按时间顺序计算 $\boldsymbol{h}_1 \to \boldsymbol{h}_2 \to \cdots \to \boldsymbol{h}_T$，GPU 的算力白白浪费
2. **长距离依赖困难**：位置 i 要影响位置 j 必须“逐跳”传递 $|i-j|$ 次，信息衰减严重
3. **固定信息瓶颈**：在 seq2seq 里，整个源序列被压到一个 $\boldsymbol{h}_T$，长句子必然丢失信息

2017 的革命

“Attention is All You Need”（Vaswani et al., 2017）提出 Transformer，**一举解决上面三个痛点**。今天 ChatGPT、BERT、GPT-4、ViT、AlphaFold、Stable Diffusion 全都建立在 Transformer 之上。

一句话目标

从 RNN 的三个痛点出发，**一步步推出**下面这个开启了 LLM 时代的公式。

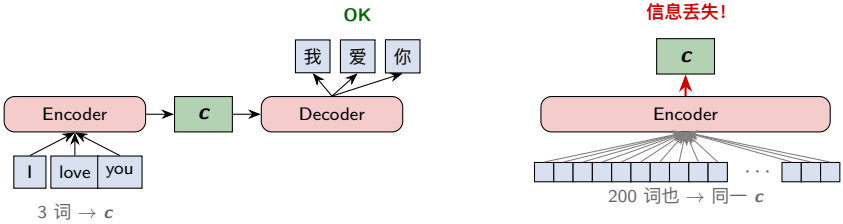
$$\boxed{\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_k}}\right)\boldsymbol{V}}$$

学完本讲你能

- 解释 Q/K/V 的物理含义，理解 “软查询” 的直觉
- 推导缩放点积注意力公式，说明为什么除以 $\sqrt{d_k}$
- 在草稿纸上手算一个 $n = 3$ 的 Self-Attention 完整过程
- 画出 Transformer Encoder 和 Decoder 的完整结构
- 说出 Self-Attention 相比 RNN 的三大优势与一个代价

机器翻译的标准建模（RNN 时代）

Encoder 读入源语言，把全部信息压到一个上下文向量 c；**Decoder** 读 c 逐词生成目标语言。

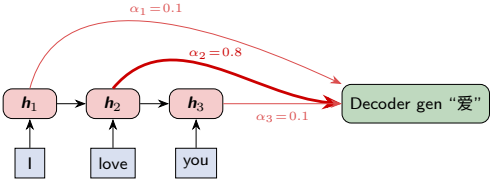


瓶颈

无论源句子多长，都要压到同一维度的 c！ 3 词和 200 词用同样宽度的管子输送信息 → 长句子必然损失。这对应 RNN 痛点 3。

核心思想（开卷 vs 闭卷）

- **无注意力 = 闭卷考试**：必须把全书内容记到脑子里（一个 $\boldsymbol{c}$）
- **有注意力 = 开卷考试**：脑中只记“主题”，遇到具体题目**回去翻书**找相关页



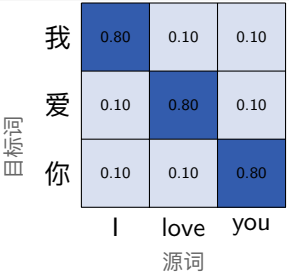
关键机制

保留所有源词的隐状态 $\boldsymbol{h}_1, \boldsymbol{h}_2, \boldsymbol{h}_3$。解码器生成每一个目标词时，**动态地算一组权重** α_i 决定往哪看。

生成“爱”时，权重聚焦在“love”（α_2 最大）→ 信息按需取用，不再有瓶颈。

可视化解读

每生成一个目标词，都会在源词上产生一组权重分布（行：目标词；列：源词）。颜色越深表示权重越大。



观察

生成“我”时，权重集中在“I”；生成“爱”时集中在“love”；生成“你”时集中在“you”。**注意力学到了一个近似单调对齐**——这正是翻译任务的真实语义。

数据库查询（硬查询）

你搜数据库里一张图：

- 你提供 **Query**：“猫的图片”
- 每条记录有 **Key**：图片标签
- 每条记录有 **Value**：图片数据

系统**只挑最相似的一个** Key 对应的 Value 返回。

硬查询不可微

argmax 是**不可微**的——反向传播无法经过它。

注意力 = 软查询

- Query $\boldsymbol{q}$：我要找什么
- Keys $\{\boldsymbol{k}_i\}$：每条记录的“标签”
- Values $\{\boldsymbol{v}_i\}$：每条记录的“内容”

按相似度**加权平均**所有 Value：

$$\text{输出} = \sum_i \alpha_i \boldsymbol{v}_i$$

其中 $\alpha_i \propto \text{sim}(\boldsymbol{q}, \boldsymbol{k}_i)$。

软查询可微

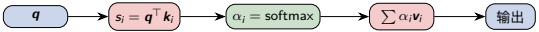
softmax + 加权平均全都可微 → 能用反向传播端到端训练。

输入

一个 Query 向量 $\boldsymbol{q} \in \mathbb{R}^{d_k}$；$n$ 个 Key-Value 对 $\{(\boldsymbol{k}_i, \boldsymbol{v}_i)\}_{i=1}^{n}$，其中 $\boldsymbol{k}_i \in \mathbb{R}^{d_k}, \boldsymbol{v}_i \in \mathbb{R}^{d_v}$。

三步走

1. **打相似度分数**（点积）：$s_i = \boldsymbol{q}^\top \boldsymbol{k}_i, \quad i = 1, \ldots, n$
2. **Softmax 归一化**：$\alpha_i = \dfrac{\exp(s_i)}{\sum_{j=1}^{n} \exp(s_j)}$，保证 $\sum_i \alpha_i = 1$
3. **加权求和**：$\mathrm{Attn}(\boldsymbol{q}, \{\boldsymbol{k}_i\}, \{\boldsymbol{v}_i\}) = \sum_{i=1}^{n} \alpha_i \boldsymbol{v}_i$



关键观察

整个计算图只有一次加权平均和一次 softmax——极简、全可微。这就是注意力的全部。

缩放点积注意力 | 矩阵形式 + 张量形状追踪

多个 Query 同时处理

实际中我们通常一次处理 n 个 Query：把它们按行堆叠成矩阵 $\boldsymbol{Q} \in \mathbb{R}^{n \times d_k}$。Keys 和 Values 同理堆叠。公式变成：

最终公式（最重要的一页）

$$\boxed{\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_k}}\right)\boldsymbol{V}}$$

softmax 是**逐行**做的（每一行是一个 Query 对所有 Keys 的权重分布）。

张量	形状	含义	典型值
$\boldsymbol{Q}$	$n \times d_k$	n 个 Query	$n = 3, d_k = 2$
$\boldsymbol{K}$	$m \times d_k$	m 个 Key	$m = 3, d_k = 2$
$\boldsymbol{V}$	$m \times d_v$	m 个 Value	$m = 3, d_v = 2$
$\boldsymbol{Q}\boldsymbol{K}^\top$	$n \times m$	分数矩阵	3×3
$\text{softmax}(\cdot/\sqrt{d_k})$	$n \times m$	权重矩阵	每行和为 1
输出	$n \times d_v$	每 Query 一个输出	3×2

自检

如果你能说清每一步的形状，你就抓住了注意力的一半。剩下一半是数值——我们马上手算一遍。

缩放点积注意力 | 为什么除以 $\sqrt{d_k}$?

现象

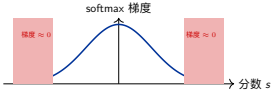
当 d_k 很大时，$\boldsymbol{q}^\top \boldsymbol{k} = \sum_{i=1}^{d_k} q_i k_i$ 的**方差会变得非常大**。

方差推导

设 q_i, k_i 相互独立、零均值、单位方差。则：

$$\mathrm{Var}(\boldsymbol{q}^\top \boldsymbol{k}) = \mathrm{Var}\left(\sum_i q_i k_i\right) = \sum_i \mathrm{Var}(q_i k_i) = \sum_i 1 = d_k$$

标准差是 $\sqrt{d_k}$。在 Transformer-base 里 $d_k = 64$，标准差约为 8——点积值在 ± 10 以上很常见。



后果与解决

点积过大 → softmax 输入过大 → 进入**饱和区**（梯度几乎为零）→ 训练停滞。
除以 $\sqrt{d_k}$ 让分数方差回到 1，softmax 工作在敏感区。这就是 “**Scaled** Dot-Product” 里的 “Scaled”。

设置

$n = 3$ 词：“I, love, you”，词嵌入维度 $d_{\text{model}} = 4$，$d_k = d_v = 2$。

词嵌入矩阵 $\boldsymbol{X} \in \mathbb{R}^{3\times 4}$

$$\boldsymbol{X} = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{pmatrix} \begin{array}{l} \leftarrow \text{I} \\ \leftarrow \text{love} \\ \leftarrow \text{you} \end{array}$$

投影权重 $\boldsymbol{W}_Q, \boldsymbol{W}_K, \boldsymbol{W}_V \in \mathbb{R}^{4\times 2}$

$$\boldsymbol{W}_Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{pmatrix}, \quad \boldsymbol{W}_K = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 0 \\ 0 & 0 \end{pmatrix}, \quad \boldsymbol{W}_V = \begin{pmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}$$

逐行算 $\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}_Q, \boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}_K, \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}_V$

$$\boldsymbol{Q} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix} \quad \boldsymbol{K} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix} \quad \boldsymbol{V} = \begin{pmatrix} 1 & 2 \\ 1 & 0 \\ 2 & 1 \end{pmatrix}$$

手算示范（第一行 $\boldsymbol{q}_1$）

$\boldsymbol{q}_1 = \boldsymbol{x}_1 \boldsymbol{W}_Q = (1, 0, 1, 0)\, \boldsymbol{W}_Q$

$= (1\cdot 1 + 0 + 1\cdot 0 + 0,\ 1\cdot 0 + 0 + 1\cdot 0 + 0)$

$= (1, 0)$

其他行类推。课后请自己验算 $\boldsymbol{K}, \boldsymbol{V}$。

分数矩阵 $\boldsymbol{S} = \boldsymbol{Q}\boldsymbol{K}^\top/\sqrt{d_k}$

$$\boldsymbol{Q}\boldsymbol{K}^\top = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 2 \end{pmatrix}$$

$$\boldsymbol{S} = \boldsymbol{Q}\boldsymbol{K}^\top/\sqrt{2} \approx \begin{pmatrix} 0 & 0.71 & 0.71 \\ 0.71 & 0 & 0.71 \\ 0.71 & 0.71 & 1.41 \end{pmatrix}$$

手算示范（s_{13}）

$s_{13} = \boldsymbol{q}_1 \cdot \boldsymbol{k}_3/\sqrt{2}$
$= (1,0)\cdot(1,1)/\sqrt{2} = 1/\sqrt{2} \approx 0.71$

权重矩阵 $\boldsymbol{A} = \mathrm{softmax}(\boldsymbol{S})$（逐行）

$$\boldsymbol{A} \approx \begin{pmatrix} 0.198 & 0.401 & 0.401 \\ 0.401 & 0.198 & 0.401 \\ 0.248 & 0.248 & 0.503 \end{pmatrix}$$

每行和为 1。

	I	love	you
I	0.20	0.40	0.40
love	0.40	0.20	0.40
you	0.25	0.25	0.50

“you” 对自己
权重 0.50（最大）

最后一步：$\boldsymbol{y} = \boldsymbol{A}\boldsymbol{V}$

把权重矩阵 $\boldsymbol{A}$ 乘到 Value 矩阵 $\boldsymbol{V}$，得到每个位置的新表示。

矩阵乘法

$$\boldsymbol{y} = \boldsymbol{A}\boldsymbol{V} \approx \begin{pmatrix} 0.198 & 0.401 & 0.401 \\ 0.401 & 0.198 & 0.401 \\ 0.248 & 0.248 & 0.503 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 1 & 0 \\ 2 & 1 \end{pmatrix} \approx \begin{pmatrix} 1.401 & 0.797 \\ 1.401 & 1.203 \\ 1.503 & 1.000 \end{pmatrix}$$

手算示范（第一行 $\boldsymbol{y}_1$）

$$\boldsymbol{y}_1 = 0.198 \cdot (1,2) + 0.401 \cdot (1,0) + 0.401 \cdot (2,1) \approx (1.401, 0.797)$$

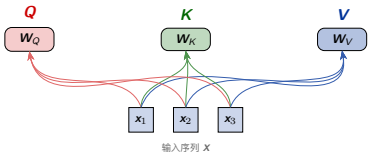
物理解读："I" 的新表示 $\boldsymbol{y}_1 = (1.401, 0.797)$ 里

- 有 19.8% 来自自己（"I" 的 Value $(1,2)$）
- 有 40.1% 来自 "love"（Value $(1,0)$）
- 有 40.1% 来自 "you"（Value $(2,1)$）

每个词的新表示都是全体词 Value 的软混合——这就是注意力的本质。

回看我们刚才的手算

刚才的例子里，Q、K、V 都是从**同一个输入序列**（“I love you”）投影出来的——这正是 **Self-Attention**！



公式改写

对一个序列 $\boldsymbol{X} \in \mathbb{R}^{n \times d_{\text{model}}}$：

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}_Q, \quad \boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}_K, \quad \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}_V$$

$$\text{Self-Attn}(\boldsymbol{X}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_k}}\right)\boldsymbol{V}$$

关键性质

每个位置的输出都“看到”了整个序列（包括自己）→ 长距离依赖变成 1 步。**所有位置并行计算** → 不再受 RNN 串行限制。

复用帧 10-12 的全部数值

前面已经把 Q/K/V、分数、权重、输出都算完了——那就是 Self-Attention！

三个位置各自 “看到了什么”（从权重矩阵 $\boldsymbol{A}$ 读出）

- **位置 1 (“I”)**：自己 0.198；“love” 0.401；“you” 0.401
  $\rightarrow$ 新表示 $\boldsymbol{y}_1 = (1.401, 0.797)$
- **位置 2 (“love”)**：“I” 0.401；自己 0.198；“you” 0.401
  $\rightarrow$ 新表示 $\boldsymbol{y}_2 = (1.401, 1.203)$
- **位置 3 (“you”)**：“I” 0.248；“love” 0.248；自己 0.503（最大）
  $\rightarrow$ 新表示 $\boldsymbol{y}_3 = (1.503, 1.000)$

和 RNN 相比

在 RNN 里，$\boldsymbol{y}_1$（最早位置）只能看到自己；$\boldsymbol{y}_3$ 要看到 $\boldsymbol{y}_1$ 的信息必须经过 $\boldsymbol{h}_1 \rightarrow \boldsymbol{h}_2 \rightarrow \boldsymbol{h}_3$ 两步。
在 Self-Attention 里，任何位置到任何位置都是一次加权平均——距离永远是 1。

从输入 $\boldsymbol{X}$ 到输出 $\boldsymbol{y}$ 的完整流水线（带 batch 维）

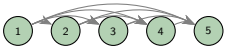
步骤	张量形状	操作（GPU 并行度）
输入 $\boldsymbol{X}$	(B, n, d)	——
投影 $\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}_Q$	$(B, n, d) \to (B, n, d_k)$	沿 B, n 全并行
投影 $\boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}_K$	$(B, n, d) \to (B, n, d_k)$	沿 B, n 全并行
投影 $\boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}_V$	$(B, n, d) \to (B, n, d_v)$	沿 B, n 全并行
分数 $\boldsymbol{Q}\boldsymbol{K}^\top$	$(B, n, d_k) \times (B, d_k, n) \to (B, n, n)$	batched matmul
缩放 $/\sqrt{d_k}$	(B, n, n)	逐元素并行
Softmax (逐行)	$(B, n, n) \to (B, n, n)$	沿最后一维
加权 $\boldsymbol{A}\boldsymbol{V}$	$(B, n, n) \times (B, n, d_v) \to (B, n, d_v)$	batched matmul
输出 $\boldsymbol{y}$	(B, n, d_v)	形状和输入对齐

关键观察

只有 softmax 是 “非矩阵乘法”，其余全是可以高度并行的矩阵乘法（GEMM）。这正是 Transformer 对 GPU 友好的根本原因。

属性	RNN/LSTM	Self-Attention
位置 i 与 j 的最短依赖路径	$\lvert i-j\rvert$ 步	**1 步**
并行性	必须串行	**完全并行**
长距离梯度	易消失/爆炸	**直通**
单层时间复杂度	$O(nd^2)$	$O(n^2d)$

RNN: $1 \to 5$ 要 4 步



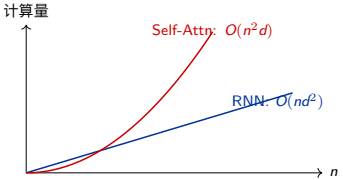
Self-Attn: 任意两点 1 步

梯度视角

在 Self-Attention 里，位置 i 和位置 j 之间**永远有一条直接的梯度路径**（经 α_{ij}）。这一点解释了为什么 Transformer 能堆到几十层而不梯度消失。

$O(n^2)$ 的代价

Self-Attention 分数矩阵 $\boldsymbol{QK}^\top$ 的形状是 $n \times n$——**序列变长 10 倍，计算量和显存都变 100 倍。**



为什么 GPU 仍然喜欢

- RNN 虽然总计算量小，但**无法并行** → GPU 算力白白闲置
- Self-Attn 虽然 n^2，但**全部是矩阵乘法** → 可以打满 GPU
- 在 $n \leq$ 几千时，GPU 实测 Self-Attn 往往**更快**

现代优化（一句话提及）

长序列（如 GPT-4 的 128K 上下文）需要特殊优化：**Flash-Attention**（内存感知）、**稀疏注意力**、**线性注意力**——但核心思路仍是把 n^2 做得更便宜。

单头的局限

一组 $(\boldsymbol{W}_Q, \boldsymbol{W}_K, \boldsymbol{W}_V)$ 只定义了**一种**相似度度量——它只能捕捉**一种**关联模式。

语言中的关联是多样的

考虑句子 "The cat that the dog chased ran away."

- **语法关系**："ran" 的主语是 "cat"（不是 "dog"）
- **语义关联**："cat"、"dog"、"chased" 都涉及动物行为
- **指代关系**："that" 指向 "cat"
- **位置关联**：相邻词有高相关

多头的想法

用 h 组**不同的** $(\boldsymbol{W}_Q^{(i)}, \boldsymbol{W}_K^{(i)}, \boldsymbol{W}_V^{(i)})$ 并行计算 h 个注意力，各学一种模式，最后拼起来。
类比 CNN 的多通道卷积——不同的卷积核学不同的模式。

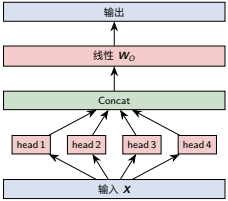
公式

对 h 个并行的头：

$$\text{head}_i = \text{Attention}(\boldsymbol{Q}\boldsymbol{W}_Q^{(i)},\ \boldsymbol{K}\boldsymbol{W}_K^{(i)},\ \boldsymbol{V}\boldsymbol{W}_V^{(i)}),\quad i = 1, \ldots, h$$

拼接后再经过一次线性投影 $\boldsymbol{W}_O$：

$$\text{MultiHead}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\ \boldsymbol{W}_O$$



参数量保持不变的巧思

典型配置：$h = 8$, $d_{\text{model}} = 512$，每头维度 $d_k = d_v = 512/8 = 64$。
每头的投影矩阵**变小**（512×64），总参数量近似等于一个单头版本。**换取** h 种不同视角。

设置

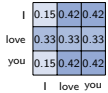
同一句 “I love you”，$h = 2$ 头，每头维度 $d_{\text{head}} = 1$（极简便于演示）。两组不同的 $\boldsymbol{W}$ 会让两头看到不同的关联。

头 1 的注意力权重

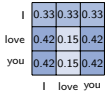
$$\boldsymbol{A}^{(1)} \approx \begin{pmatrix} 0.155 & 0.422 & 0.422 \\ 0.333 & 0.333 & 0.333 \\ 0.155 & 0.422 & 0.422 \end{pmatrix}$$

头 2 的注意力权重

$$\boldsymbol{A}^{(2)} \approx \begin{pmatrix} 0.333 & 0.333 & 0.333 \\ 0.422 & 0.155 & 0.422 \\ 0.422 & 0.155 & 0.422 \end{pmatrix}$$



头 1：偏向 love, you



头 2：偏向 I, you

观察

两个头关注了序列的**不同子集**。最后 concat + $\boldsymbol{W}_O$ 把两种视角融合成输出。真实 Transformer 的头会学到语法、指代、语义等可解释的模式。

一个 Encoder Block 由 4 步组成

1. **Multi-Head Self-Attention**
2. **Add & Norm**（残差 + LayerNorm）
3. **Feed-Forward Network**（两层 MLP）
4. **Add & Norm**

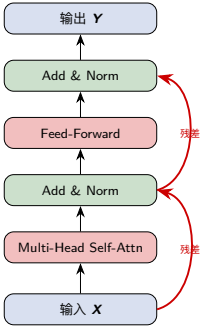
数学形式

$$\boldsymbol{Z} = \mathrm{LN}(\boldsymbol{X} + \mathrm{MHA}(\boldsymbol{X}))$$

$$\boldsymbol{Y} = \mathrm{LN}(\boldsymbol{Z} + \mathrm{FFN}(\boldsymbol{Z}))$$

关键结构

两次 “子层 + 残差 + LN”。这个结构会被堆 N 次 (Transformer-base: $N = 6$)。



为什么需要残差（$\boldsymbol{X}$ + 子层($\boldsymbol{X}$)）

- Transformer-base: 6 层；GPT-3: 96 层；LLaMA: 32+ 层
- **残差保证梯度直通**，让深堆叠可行（和 ResNet 同理）
- 子层可以被视作对 $\boldsymbol{X}$ 的**修正**，而不是重建

LayerNorm 公式

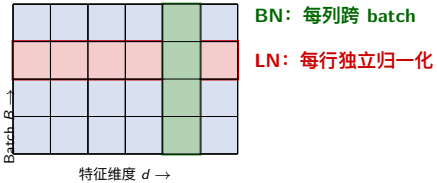
对每个位置的特征向量 $\boldsymbol{a} \in \mathbb{R}^d$：

$$\mu = \frac{1}{d}\sum_i a_i, \quad \sigma^2 = \frac{1}{d}\sum_i (a_i - \mu)^2$$

$$\mathrm{LN}(\boldsymbol{a}) = \gamma \odot \frac{\boldsymbol{a}-\mu}{\sqrt{\sigma^2+\epsilon}} + \beta$$

LayerNorm vs BatchNorm（方向不同）

每行 = 一个样本在所有特征维度上的值（Batch B 行，特征 d 列）。



为什么 Transformer 用 LN

- BN 需要大 batch，对序列变长不友好
- LN 对每个样本**独立**计算，与 batch 无关
- 推理时 batch 可以 =1，LN 完全不受影响

公式

$$\mathrm{FFN}(\boldsymbol{x}) = \max(0,\ \boldsymbol{x}\boldsymbol{W}_1 + \boldsymbol{b}_1)\boldsymbol{W}_2 + \boldsymbol{b}_2$$

就是一个两层 MLP + ReLU，中间层更宽。

典型尺寸（Transformer-base）

$$\boxed{d = 512} \xrightarrow{\boldsymbol{W}_1 + \mathrm{ReLU}} \boxed{d_{ff} = 2048} \xrightarrow{\boldsymbol{W}_2} \boxed{d = 512}$$

中间层宽度 $d_{ff} = 4d$ 是惯例。参数量：$d \cdot 4d + 4d \cdot d = 8d^2$。

为什么 Transformer 还需要 FFN

- Self-Attention 本质是**加权平均**（线性操作 + softmax）
- 没有 FFN 的话，Transformer 的非线性能力非常有限
- FFN 是 Transformer 的 “主要参数容器”——**最新研究发现它起到了类似 “知识存储” 的作用**
- FFN **逐位置独立**（同一个 MLP 作用在所有位置上，位置间不交互）

问题：Self-Attention 对顺序不敏感

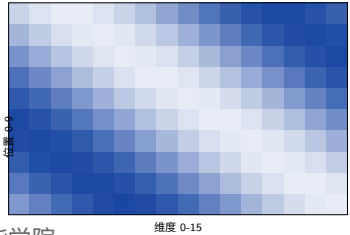
“我爱你” 和 “你爱我” 在 Self-Attn 眼里**完全一样**（同一个词集）——因为 Self-Attn 是置换等变的。
必须以某种方式**告诉模型位置信息**。

解决：给词嵌入加上位置向量

$$x'_i = \text{Embed}(\text{word}_i) + \text{PE}_i$$

原版 Transformer 的 PE（sin/cos）：

$$\text{PE}_{\text{pos},2k} = \sin\left(\frac{\text{pos}}{10000^{2k/d}}\right), \quad \text{PE}_{\text{pos},2k+1} = \cos\left(\frac{\text{pos}}{10000^{2k/d}}\right)$$



设计的巧妙之处

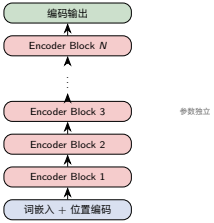
- 每个位置的 PE **唯一**
- 相对位置可由**线性变换**得到
- 能**外推**到训练时没见过的长度

现代变体（只提一句）

- **可学习 PE**：BERT、ViT 用
- **RoPE**（旋转位置编码）：LLaMA、GPT 系列用
- **ALiBi**：通过注意力偏置注入位置

堆叠方式

把 N 个 Encoder Block **前后串联**，前一层的输出是后一层的输入。所有层结构相同但**参数独立**。



典型模型规模

模型	N（层）	d_{model}	h（头）	参数量	发布
Transformer-base	6	512	8	65M	2017
BERT-base	12	768	12	110M	2018
GPT-3	96	12288	96	175B	2020
LLaMA-2-70B	80	8192	64	70B	2023

为什么 Decoder 和 Encoder 不一样?

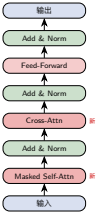
Encoder 做**理解**：把源序列并行压成上下文向量（双向可见，一次算完）。
Decoder 做**生成**：拿上下文一个词一个词吐出目标（自回归）→ 必须多出两种能力：只看已生成的过去（因果）+ 能查询源端（对齐）。

Encoder Block（回顾）

1. Multi-Head Self-Attn
2. Add & Norm
3. Feed-Forward
4. Add & Norm

Decoder Block（多了什么）

1. **Masked** Multi-Head Self-Attn （看过去）
2. Add & Norm
3. **Cross-Attention** （查源端）
4. Add & Norm
5. Feed-Forward
6. Add & Norm

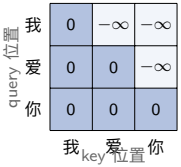


两大新增 = 两种 "看"

Masked Self-Attn：Q/K/V 都来自目标端，加因果掩码 → 目标端只看已生成的自己。
Cross-Attn：Q 来自目标端，K/V 来自 *Encoder* 输出 → 目标端查询源端。

核心矛盾

训练时想并行（一次 forward 算全部位置的 loss）；**推理**时必须串行（生成第 t 词时还没有 $t+1$）。
掩码就是让"并行训练"伪装成"串行推理"的魔法：softmax 前把不该看的位置打 $-\infty$，对应权重就变 0。



下三角 = 因果

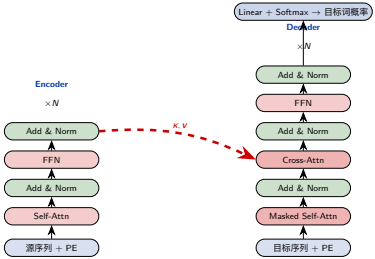
第 t 个 query 只能看 $\leq t$ 的 key——这就是**自回归**的数学表达。**GPT 全家都用这个掩码。**

训练（教师强制）vs 推理（自回归）

	训练	推理
输入	整句"我爱你"并行喂入	逐步累加"我"→"我爱"
并行度	3 个位置一次算完	必须循环 3 步
监督信号	整行 label（"爱 你 [EOS]"）	只取下一词
偷看未来	掩码强制禁止	物理上根本不存在

一句话

掩码让训练"假装"在做推理——因此同一套参数，既能并行高效训练，又能串行自回归生成。



三种注意力的 Q / K / V 来源（最关键的对比）

层	Q 来自	K, V 来自
Encoder Self-Attn	源端	源端
Decoder Masked Self-Attn	目标端	目标端（仅过去）
Cross-Attn	**目标端**	**Encoder 输出**

翻译直觉：对齐

生成“爱”时：Q = “爱”的 Decoder 表示；K/V 来自源端“I love you”。

Cross-Attn 权重会**自动聚焦到“love”** → 这就是传统机器翻译里**“对齐”**被神经网络自动学出来的方式。

设定

Encoder 已把源端 “I love you” 编码为 $\boldsymbol{H}^{\text{enc}} \in \mathbb{R}^{3\times d_{\text{model}}}$（算一次，后续 Decoder 每步复用）。**Decoder** 从 [BOS] 出发，逐词预测，直到输出 [EOS]。

推理：三步生成

步	Decoder 已知输入	Masked Self-Attn 可见	Cross-Attn 聚焦源端	预测
1	[BOS]	{[BOS]}	“I”	我
2	[BOS] 我	{[BOS], 我}	“love”	爱
3	[BOS] 我爱	{[BOS], 我, 爱}	“you”	你

每步 Decoder 内部

1. **Masked Self-Attn**：Q 查已生成序列的 K/V
2. **Cross-Attn**：Q 来自目标端，K/V 来自 $\boldsymbol{H}^{\text{enc}}$
3. **FFN + Linear + Softmax**：出下一词分布

训练为何能并行？

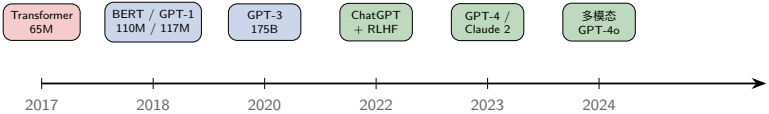
把 “[BOS] 我爱你” **一次性**喂进 Decoder，**下三角掩码**让第 t 位看不到 $> t$ 的未来——等价于并行模拟上表 3 步，**一次 forward 出全部 loss**。

```python
import torch, torch.nn as nn, math

class SelfAttention(nn.Module):
    """从零实现的 Scaled Dot-Product Self-Attention。"""
    def __init__(self, d_model, d_k):
        super().__init__()
        self.W_q = nn.Linear(d_model, d_k, bias=False)
        self.W_k = nn.Linear(d_model, d_k, bias=False)
        self.W_v = nn.Linear(d_model, d_k, bias=False)
        self.scale = math.sqrt(d_k)

    def forward(self, x, mask=None):
        # x: (B, n, d_model)
        Q = self.W_q(x)                                  # (B, n, d_k)
        K = self.W_k(x)                                  # (B, n, d_k)
        V = self.W_v(x)                                  # (B, n, d_k)
        scores = Q @ K.transpose(-2, -1) / self.scale    # (B, n, n)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, float("-inf"))
        attn = torch.softmax(scores, dim=-1)             # (B, n, n)
        return attn @ V                                  # (B, n, d_k)

# 官方版（实际工程直接用这个，自动支持多头）
mha = nn.MultiheadAttention(embed_dim=512, num_heads=8, batch_first=True)
x = torch.randn(2, 10, 512)                              # (B=2, n=10, d=512)
out, attn_weights = mha(query=x, key=x, value=x)         # self-attention
print(out.shape, attn_weights.shape)                     # (2,10,512), (2,10,10)
```

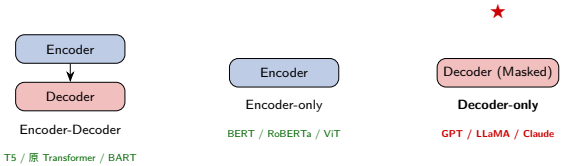


规模的飙升

模型	参数量	年份	关键创新
Transformer (base)	65M	2017	Self-Attention 架构
BERT-base	110M	2018	双向预训练 + 微调
GPT-3	175B	2020	规模化 + 零/少样本学习
ChatGPT	∼200B	2022	RLHF 对齐
GPT-4 / Claude 2	∼ 万亿级	2023	多模态 + 推理

观察

2017 年的那一页公式，7 年内把参数量从 10^8 推到了 10^{12}，并发明了**和人聊天的机器**。所有突破都是在**同一块积木**（Transformer）上做大做深 + 更多数据 + RLHF。



为什么 Decoder-only 赢了？

- **任务统一**：任何任务都能表述为“续写”（翻译、问答、摘要、编程……）
- **训练简单**：一个目标——预测下一个 token；语料就是整个互联网的文本
- **In-context learning**：只靠 prompt 就能学新任务，不需要微调
- **工程简单**：没有 Encoder-Decoder 之间的耦合

承前启后

下节课（L10）我们详细讲**预训练-微调范式**：如何把 Transformer 三种范式用在 BERT / GPT / ViT 上。今天你已经具备了理解它们的全部基础。

三大要点

1. **注意力 = 软查询**：按 Query 与 Key 的相似度对 Value 加权平均，全程可微
2. **Self-Attention**：让每个位置直接看到所有位置（含自己）——**并行 + 长距离依赖天然解决**
3. **Transformer = Self-Attn + FFN + 残差 + LN + 位置编码**（外加 Decoder 的 Masked + Cross-Attn）

最重要的一个公式

$$\boxed{\mathrm{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \mathrm{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_k}}\right)\boldsymbol{V}}$$

思想转折

从 “如何记住过去”（RNN）到 “如何检索过去”（Attention）。
从 “把历史压进一个向量” 到 “保留全部历史、按需取用”。
这是序列建模的**范式转变**，也是 LLM 时代的出发点。

下一讲（第 10 讲）：预训练范式 BERT/GPT/ViT

- **预训练-微调**的核心思想与自监督任务
- **BERT**（Encoder-only）：双向编码 + 掩码语言模型（MLM）
- **GPT**（Decoder-only）：自回归 + In-Context Learning
- **ViT**（Encoder-only）：把 Transformer 用到图像
- HuggingFace 实战：加载预训练模型

课后练习（4 题）

1. 用纸笔把本讲的贯穿手算例从 $\boldsymbol{X}$ 完整推到输出 $\boldsymbol{y}$，检查是否和讲义一致
2. 证明：若 $\boldsymbol{q}, \boldsymbol{k}$ 每维独立、零均值、单位方差，则 $\mathrm{Var}(\boldsymbol{q}^\top \boldsymbol{k}) = d_k$；由此推导为什么除以 $\sqrt{d_k}$ 而不是 d_k
3. 用 `nn.MultiheadAttention` 替代第 8 讲的 GRU 做 IMDB 分类，对比准确率和训练速度
4. 阅读 “Attention is All You Need”（Vaswani et al., 2017）论文的图 1，把图上每个模块和今天的帧 28 对应起来

谢谢！

问题与讨论

下次课：BERT、GPT 与 ViT 预训练模型

孟翰　人工智能学院